# Quiz: Containers

**Total Score:** $/2^3$

**Printed Name:**

---

**Quiz rules:**

1. You MAY use any printed or handwritten notes.
2. You MAY NOT use a computer or any other electronic device.

**Problem 1.** Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
class Range:
    def __init__(self, start, stop, step=1):
        self.start = start
        self.stop = stop
        self.step = step

    def __len__(self):
        return max(0, -((self.start - self.stop) // self.step))

    def __contains__(self, x):
        if (x - self.start) % self.step != 0:
            return False
        return 0 <= (x - self.start) // self.step < len(self)

    def __getitem__(self, i):
        n = len(self)
        if i >= n:
            raise IndexError(i)
        return self.start + self.step * i

try:
    r = Range(0, 20, 3)
    n = len(r)
    print('n =', n)
    x = r[-1]
    print('x =', x)
except ValueError:
    print('ValueError')
except TypeError:
    print('TypeError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 10 / 16 = 0.62

**Problem 2.** Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
class Node:
    def __init__(self, val, nxt=None):
        self.val = val
        self.nxt = nxt

    def __len__(self):
        if self.nxt is None:
            return 1
        return 1 + len(self.nxt)

    def __contains__(self, target):
        if self.val == target:
            return True
        if self.nxt is None:
            return False
        return target in self.nxt

    def __getitem__(self, i):
        if i == 0:
            return self.val
        if self.nxt is None:
            raise IndexError(i)
        return self.nxt[i]

try:
    xs = Node(1, Node(2, Node(3, Node(4, Node(5)))))
    n = len(xs)
    print('n =', n)
    v = xs[2]
    print('v =', v)
except IndexError:
    print('IndexError')
except TypeError:
    print('TypeError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: $7 / 16 = 0.44$

**Problem 3.** Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
class Node:
    def __init__(self, val):
        self.val = val
        self.left = None
        self.right = None

def insert(t, val):
    if t is None:
        return Node(val)
    if val < t.val:
        t.right = insert(t.right, val)
    elif val > t.val:
        t.left = insert(t.left, val)
    return t

def postorder(t):
    if t is None:
        return []
    return postorder(t.left) + postorder(t.right) + [t.val]

class Tree:
    def __init__(self):
        self.root = None

    def insert(self, val):
        self.root = insert(self.root, val)

    def __getitem__(self, i):
        return postorder(self.root)[i]

try:
    t = Tree()
    for val in [5, 7, 3, 9, 1]:
        t.insert(val)
    x = t[1]
    print('x =', x)
    y = t[-1]
    print('y =', y)
except IndexError:
    print('IndexError')
except TypeError:
    print('TypeError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 11 / 16 = 0.69

**Problem 4.** Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
class FooBar:
    def __init__(self, x, y=None):
        self.x = x
        self.y = y

    def wob(self, n):
        if n == 0:
            return 0
        return self.x + self.y.wob(n - 1)

t = FooBar(2)
u = FooBar(3)
v = FooBar(5)
t.y = u
u.y = v
v.y = t
try:
    print(t.y.y.x)
    print(t.wob(7))
except RecursionError:
    print('RecursionError')
except AttributeError:
    print('AttributeError')
except TypeError:
    print('TypeError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: $9 \;/\; 16 = 0.56$

## LLM Model Performance

